Prerequisites and objectives

- Prerequisites are the dictionary ADT, modular arithmetic, expectation, linked lists, and amortized analysis.
- Separate hash functions, collision resolution, and resizing policies. State the assumptions behind each performance claim.
- Trace chaining, linear probing, tombstone deletion, and rehashing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dictionary ADT and collisions

- A dictionary maps each unique key to a value. Operations include find, insert-or-update, and erase.
- A hash function $h : U \to \{0, \ldots, m-1\}$ maps a key universe into m buckets. Collisions are unavoidable when $|U| > m$.
- Equal keys must have equal hashes. The converse fails. Different colliding keys still need equality comparison.
- A hash is not a unique identifier. Comparing hashes without keys turns a collision into a data error.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Hash functions and probability spaces

- The division method is h(k)=k mod m. Keys with matching modular structure may all enter one bucket.
- Randomly selecting a function differs from assuming uniform input. A universal family requires collision probability at most 1/m for every distinct fixed x, y, over the random choice of h.
- If input is fixed before h is selected, universal hashing bounds expected collisions through internal randomness. Adaptive adversaries may require stronger assumptions.
- Cryptographic hashing and ordinary hash tables address different threat models. Output bit length alone does not establish resistance to attacks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Separate chaining and load factor

- Each bucket stores a list or another collection of keys sharing a hash. Load factor $\alpha = n/m$ can exceed 1.
- With uniform hashing, an independent unsuccessful query has expected bucket length α. For a universal collision bound and a fixed query key outside a fixed stored set, the expectation is at most α. Both give expected $O(1+\alpha)$ search under constant-time key operations.
- Successful-search expectation also depends on insertion order and query distribution. Standard uniform models give $O(1+\alpha)$.
- All n keys can still occupy one bucket, giving $\Theta(n)$ worst-case search. An expected guarantee does not eliminate the worst case.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Worked chaining example

- Let m=7, h(k)=k mod 7, and insert `10,17,24,8`.
- Appending within a bucket gives bucket 3 as `[10,17,24]` and bucket 1 as `[8]`. Other buckets are empty.
- Finding 24 scans three keys. Finding absent 31 also scans bucket 3. A small global n/m does not guarantee a short particular search.
- Tree-based buckets can support other requirements or control severe collisions. Their different implementation and costs are not automatic properties of basic chaining.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete chaining dictionary operations

```
find(k):
    j=hash(k) mod m
    for node in bucket[j]:
        if equal(node.key,k): return reference(node.value)
    return absent
insert_or_update(k,v):
    j=hash(k) mod m
    for node in bucket[j]:
        if equal(node.key,k): node.value=v; return updated
    prepend newly constructed pair(k,v) to bucket[j]; n+=1
erase(k):
    j=hash(k) mod m; predecessor=dummy_before(bucket[j])
    while predecessor.next!=null:
        if equal(predecessor.next.key,k): unlink and release; n-=1; return true
        predecessor=predecessor.next
    return false
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete chaining dictionary operations (continued)

This code prepends, unlike the earlier append-based example. After 10,17,24 enter bucket 3, the chain is [24,17,10]. Find 24 compares once; find 31 compares three times. Erasing 17 leaves [24,10]; reinserting 24 updates its value without increasing n. Both insertion orders implement the same dictionary but differ in individual successful-search costs.

A dummy predecessor may be temporary rather than stored user data. Removing the first node must update the bucket head. Expected chain length depends on the hashing model; collisions can still make worst-case search $\Theta(n)$. Skipping duplicate checks changes a unique-key dictionary into multimap semantics.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Whole-table collision pairs versus a fixed query

```
// S is fixed before the random function h is selected.
tau=0
for each unordered pair {x,y} in S:
    tau += indicator(h(x)==h(y))
E[tau] = sum_{unordered {x,y}} Pr(h(x)==h(y))
```

An unsuccessful fixed query compares one key with n stored keys, giving expected chain length at most n/m. Whole-table collision counting has $\binom{n}{2}$ unordered pairs. A universal family bounds each pair's collision probability by $1/m$, hence $\mathbb{E}[\tau] \le n(n-1)/(2m)$. Exact pair probability 1/m gives equality. Both calculations use linearity without independence among all pair events, but n/m is not the table's total collision count.

If $m = \Theta(n^{4/3})$, total expected collisions are $O(n^{2/3})$, while a fixed unsuccessful query has $\mathbb{E}[X] \le n/m = O(n^{-1/3})$, plus the unavoidable bucket-access constant. With m=Θ(n), Θ(n) total expected pairs remain compatible with expected O(1) per query. A bucket of b keys contributes b(b−1)/2 collision pairs, not b−1.

Whole-table collision pairs versus a fixed query (continued)

Worst-case expectation here means $\max_{|S|=n} \mathbb{E}_h[\tau(S,h)]$. Fix any S before choosing h. Universality gives the same bound for each such set. This differs from $\mathbb{E}_h[\max_{|S|=n} \tau(S,h)]$, which allows the input to depend on the observed function. The quantifier order changes the model. For nonnegative τ and $a > 0$, Markov's inequality gives $\Pr(\tau \geq a) \leq \mathbb{E}[\tau]/a$. A small expectation does not guarantee zero collisions for every selected function.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Constructing a universal family and rejecting false randomization

```
// p prime, keys are distinct vectors in F_p^2.
choose a,b independently and uniformly from 0..p-1
h(a,b,(x1,x2))=(a*x1+b*x2) mod p
// A random output translation of fixed g:
g_c(x)=(g(x)+c) mod m
```

For distinct vectors x,y, difference $\delta=(x1-y1,x2-y2)$ is nonzero. If $\delta1\neq0$, fixing any b leaves exactly one solution a to $a\delta1+b\delta2=0$ because nonzero field elements are invertible. Collision probability is exactly 1/p. If $\delta1=0$, then $\delta2\neq0$ and solve uniquely for b instead. The proof includes the function a=b=0; one all-colliding function does not invalidate the family's expected guarantee.

For p=11 and keys (0,0),(1,0),(0,1),(1,1), six pairs each collide under exactly 11 of 121 coefficient choices. Total expected collisions are 6/11. Function a=b=0 makes all six pairs collide, an allowed low-probability outcome within the same expectation.

Constructing a universal family and rejecting false randomization (continued)

By contrast, randomly translating a fixed g preserves collisions: g_c(x)=g_c(y) exactly when g(x)=g(y). With g(x1,x2)=x1, keys (3,1),(3,7) always collide, so this is not universal. Reducing a large integer modulo p before hashing can likewise collapse distinct keys prematurely. Encode keys $0..p^2-1$ injectively as two base-p digits rather than discarding the higher digit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

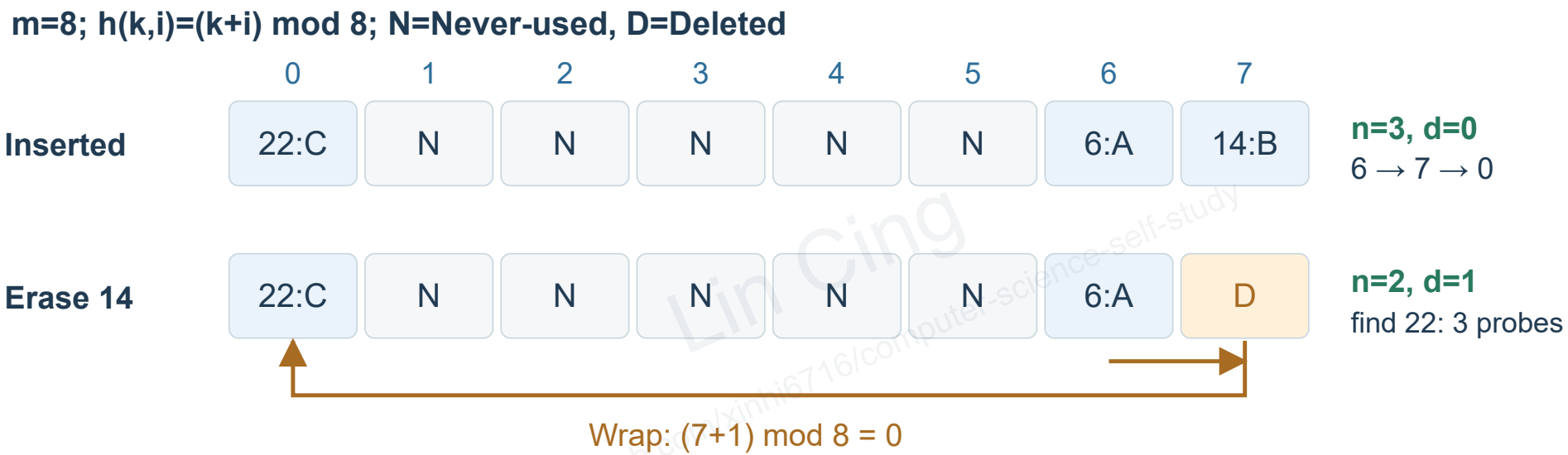
Slot states in open addressing

- Every element resides in the table. A key searches probe sequence `h(k,0),h(k,1),…`. Normally n<m is required.
- Slots are Never-used, Occupied, or Deleted. The three states have different meanings.
- Search may stop at Never-used because insertion could not have passed it to place the same key later. Deleted does not permit stopping.
- Insertion may remember the first Deleted slot but must continue checking for an existing equal key, avoiding duplicate dictionary keys.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Linear probing: a collision run wraps around

Fix m=8 and $h(k,i) = (k+i) \bmod 8$. Insert (6,A), (14,B), (22,C); every home slot is 6. Let n count live keys and d count Deleted slots. N means Never-used since the latest table initialization.



Deleting 14 changes slot 7 to D. Finding 22 still probes 6,7,0: three probes. Making slot 7 N instead would incorrectly stop after two. A D slot retains no readable old pair; it records that search must continue.

Primary clustering: different homes extend the same run

Start again from the three-key state before deletion, then insert 8 and 1. Cells show keys only; N is Never-used. Neither insertion needs growth.

Action	0	1	2	3	4	5	6	7
Initial	22	N	N	N	N	N	6	14
Insert 8: probe 0→1	22	8	N	N	N	N	6	14
Insert 1: probe 1→2	22	8	1	N	N	N	6	14

Key 8 has home 0 and key 1 has home 1, unlike the original home 6. Both extend the circular run 6→7→0→1→2. A new key with its home inside this run must scan beyond its end. Primary clustering affects more than keys with identical homes. Consecutive probing has locality, but not the probability model of an independent uniform permutation.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

The same Deleted slot, two different decisions

After erasing 14, run `insert_or_update(22,C*)`, then `insert_or_update(30,Z)`.

Update 22 → C*

① slot 6: key=6, different

② slot 7: Deleted; remember 7

③ slot 0: key=22 found

Update 0; leave candidate 7 unused

n=2, d=1; exactly one key 22

Insert 30 → Z

① slot 6: key=6, different

② slot 7: Deleted; remember 7

③ slot 0: key=22, different

④ slot 1: Never-used proves absence

Write slot 7; n=3, d=0

The update finds 22 at slot 0 and leaves candidate 7 unused. Inserting 30 reaches N at slot 1 before writing the earliest D at 7. Final slots 0,6,7 contain 22:C*,6:A,30:Z.

Lin Cing
http://github.com/xinhi6716/computer-science-self-study

Why Never-used stops search but Deleted does not

Invariant: no live key has N before its stored position along its probe sequence. Insertion uses the first N as evidence of absence. Erasing O→D creates no new N, and reusing D changes no N before another live key. These operations preserve the invariant.

Incorrect change	Probe or write outcome	Broken contract
Erase 14 by making slot 7 N	Find 22 stops at 6→7, missing slot 0	Reachability of a live key
Update 22 by writing at the first D	Slots 7 and 0 both contain key 22	Dictionary key uniqueness

An `insert_new` operation may skip duplicate checking if its caller proves absence. Our `insert_or_update` has no such precondition: find the key, reach N, or examine all m slots before deciding. Algorithms Chapter 11 calls this same stopping state EMPTY; it corresponds to Never-used here. Neither course stops at Deleted.

Backshift deletion is a different strategy: moves must respect homes and probe intervals and preserve reachability. It is not an in-place change from tombstone to N.

With no Never-used slot, stop after one cycle

Test the fixed-capacity `locate_or_slot` with automatic resizing and cleanup disabled. Insert 6,14,22,1,2,3,4,5 to fill eight slots, then erase all seven keys other than 22. Now n=1,d=7 and no N remains.

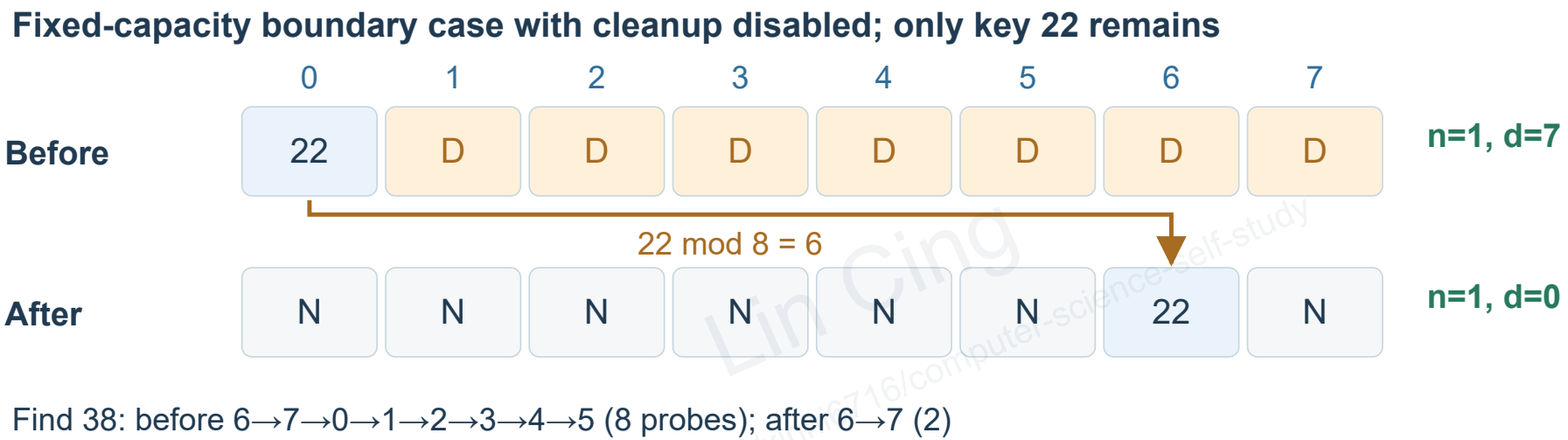
For absent key 38, home=6. Probe 6,7,0,1,2,3,4,5, remembering the first D at 6. After eight nonmatches, return absent(6); never wait indefinitely for N. If all eight slots are Occupied with other keys, return full.

This boundary tests the locator. The later automatic policy cleans earlier and does not retain d=7. **A valid fixed-capacity state need not be a state retained by that maintenance policy.**

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Same-capacity rehash: relocate live keys

Keep only key 22 from the fixed-capacity example. Allocate eight N slots and reinsert each live pair using the new table's probe sequence.



Key 22 moves from 0 to 6; n stays 1 and d becomes 0. Finding 38 now probes only 6,7. Merely clearing old D slots to N while keeping 22 at 0 would make search stop at home 6. Cleanup removes markers that searches must cross. Its cost includes scanning and reinserting live keys; the later maintenance thresholds account for that work.

Double hashing and probe coverage

- Double hashing uses $h(k,i)=(h_1(k)+i\cdot h_2(k)) \bmod m$. The step and m must be coprime to cover all m slots.
- With m=8 and step 2, only four slots of the same parity are visited. Insertion can fail despite empty slots elsewhere.
- A prime m with $1 \le h_2(k) < m$ ensures coprimality. Power-of-two capacities can use odd steps.
- Quadratic probing needs specified coefficients, capacity, and load restrictions. Writing $h+i^2$ alone does not guarantee reaching every empty slot.

Lin Cing
https://github.com/xinhi6716/computer-science-self-s

Locate or select a candidate before writing

```
locate_or_slot(key):
    first_deleted=none
    for i=0..m-1:
        j=probe(key,i)
        if state[j]==Occupied and equal(keys[j],key): return found(j)
        if state[j]==Deleted and first_deleted==none: first_deleted=j
        if state[j]==Never_used:
            return absent(first_deleted if first_deleted!=none else j)
    if first_deleted is set: return absent(first_deleted)
    return full
```

Probe at most m slots and require full coverage. Read keys[j] only for Occupied; Deleted owns no live pair. Return found, absent(slot), or full so the caller separates update, insertion, and lack of capacity.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Automatic updates: locate again after rebuilding

```
insert_or_update(k,v):
    result=locate_or_slot(k)
    if result is found(j): values[j]=v; return
    if n+1>3*m/4: rehash(2*m); result=locate_or_slot(k)
    else if deleted>m/8: rehash(m); result=locate_or_slot(k)
    j=result.slot
    if state[j]==Deleted: deleted-=1
    construct pair(k,v) at j; state[j]=Occupied; n+=1

erase(k):
    if locate_or_slot(k) is found(j):
        destroy item; state[j]=Deleted; n-=1; deleted+=1
        if m>minimum_capacity and n<m/4:
            rehash(max(minimum_capacity,m//2))
```

Handle an existing key before checking maintenance thresholds. Rehash can relocate keys, so locate again rather than reuse an old slot index. The eight-slot update/insertion trace has $d=1\le m/8$ and a live load below the growth threshold; it causes no extra rebuild. The seven-slot exercise explicitly fixes capacity and disables this layer.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

The model behind expected probe bounds

- Ideal uniform hashing assumes each key's probe permutation is uniform and independent of the prior table history. With $\alpha<1$ and no tombstones, unsuccessful search takes at most $1/(1-\alpha)$ expected probes.
- In the same model, equally likely successful queries among keys inserted without deletion have bound $(1/\alpha)\ln(1/(1-\alpha))$ for $\alpha>0$.
- At $\alpha=0.9$ the unsuccessful bound is 10. It depends on load and diverges as α approaches 1.
- These are not exact linear-probing formulas. They omit tombstones, hash computation, and long-key equality costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deriving ideal probe upper bounds

- Assume m−n Never-used slots, no tombstones, and a uniform random probe permutation independent of the occupied set. The first i probes all hit occupied slots with probability $(n)_i/(m)_i \le \alpha^i$, using falling factorials.
- For probe count X, the tail-sum identity gives $\mathbb{E}[X] = \sum_{i\ge 0} \Pr(X > i) \le \sum_{i\ge 0} \alpha^i = 1/(1-\alpha)$. Both α<1 and absence of tombstones enter this proof.
- For equally likely successful queries among n inserted, undeleted keys, average over insertion loads j/m. Since 1/(1−x) increases, its left rectangle sum is bounded by its integral: $\mathbb{E}[X] \le \frac{1}{n}\sum_{j=0}^{n-1} \frac{1}{1-j/m} \le \frac{1}{\alpha}\ln\frac{1}{1-\alpha}$.
- This derives the bound from the probability model. Linear probing's contiguous clusters do not satisfy the random-permutation premise.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Resizing must recompute placement

- Changing capacity changes the key-to-slot mapping. Reinsert existing keys rather than merely copying old slots into a larger array.
- Doubling at a fixed load threshold scans and allocates $\Theta(n)$ storage when $m=\Theta(n)$. Reinsertion also has expected $\Theta(n)$ cost under suitable random assumptions and a load bounded away from 1. It need not be worst-case linear under arbitrary collisions.
- These conditions give expected amortized $O(1)$ insertion. The claim combines probability assumptions with accounting across a sequence. It is not worst-case constant time per operation.
- Equal growth and shrink thresholds can trigger repeated rebuilding under alternating updates. Distinct thresholds avoid this oscillation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rehash relocation and expected analysis

```
rehash(new_m):
    allocate new_m Never_used slots
    choose compatible new hash parameters if required
    for each old Occupied pair (k,v):
        insert into new table using its new probe sequence
    commit new table; deleted=0; preserve n; release old table
```

For keys 10,17,24,8 with m=7, buckets are 3:[10,17,24],1:[8]. At m=14 using k mod14, buckets become 10:[10,24],3:[17],8:[8]. Old positions cannot remain unchanged. Open addressing likewise reinserts using new probe sequences and removes tombstones.

For a fixed unsuccessful query key $x \notin S$ and a fixed stored set S with $|S| = n$, define

$$X = \sum_{y \in S} \mathbf{1}[h(y) = h(x)].$$

Linearity of expectation and universality give

$$\mathbb{E}[X] = \sum_{y \in S} \Pr(h(y) = h(x)) \leq n/m.$$

Rehash relocation and expected analysis (continued)

This calculation does not need independence among collision events, but both x and S must be fixed before the function is selected. Adaptive attacks are a different model. If $x \in S$, the self-collision contributes exactly 1, giving the bucket-length bound $1 + (n-1)/m$. The unsuccessful-query bound cannot be reused in that case. A successful-query distribution concentrated on the longest chain cannot inherit the average for uniform queries.

Scanning costs Θ(m+n). When m=Θ(n), bounded load and suitable randomness make reinsertion expected Θ(n). Colliding insertions with duplicate checks may take quadratic time, so worst-case linear rehash is not unconditional. Separate 3/4 growth and 1/4 shrink thresholds plus tombstone cleanup support the combined expected-amortized update guarantee.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Cleanup-threshold counterexample and amortized accounting

```
// m starts at a power-of-two minimum_capacity>=8.
// Before inserting a genuinely absent key:
if n+1>3*m/4: rehash(2*m)
else if deleted>m/8: rehash(m)
// After deleting a present key:
if m>minimum_capacity and n<m/4:
    rehash(max(minimum_capacity,m//2))
// Rehash resets deleted=0 and reinserts all n live entries.
```

Resizing and tombstone cleanup need different triggers. The rule "rebuild at the same capacity if n+deleted+1>3m/4" fails near n=3m/4. Delete one key, giving n=3m/4−1 and deleted=1; reinserting it could reuse that tombstone, but the rule rebuilds immediately. With m=128 and 96 keys, 500 repetitions of deleting and reinserting key 95 cause 500 rebuilds and 47,500 record reinsertions. Constantly many operations cannot pay Θ(m) per repetition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Cleanup-threshold counterexample and amortized accounting (continued)

The corrected rule cleans at the same capacity only after deleted>m/8, requiring Ω(m) deletions since the previous rebuild. Lookup precedes resizing, so updating an existing value never resizes. A genuinely new key first tests live load 3/4, then the deleted threshold. After every insertion, n+deleted≤7m/8: without rebuilding, n≤3m/4 and deleted≤m/8; rebuilding clears tombstones; reusing one does not increase used slots. Deletion only converts Occupied to Deleted, preserving that sum. Consequently at least m/8 Never-used slots remain even after subsequent deletions.

Growth leaves live load about 3/8, while shrinking leaves it at most 1/2. Reaching the opposite resize threshold requires Ω(m) live-count changes, preventing one-delete/one-insert oscillation. Charge same-capacity cleanup separately to its accumulated deletions. Under the stated uniform-probing or compatible random model, each rebuild costs expected Θ(m+n), giving expected amortized O(1) probe work over updates. Worst-case collisions still allow linear operations or quadratic reinsertion. Odd double-hash steps cover power-of-two capacities, but coverage alone does not establish the ideal random-permutation model.

Overflow and engineering conditions

- In chaining, overflow means longer bucket contents. A full open-addressed table must be detected rather than probed forever.
- Hashing an L-symbol string can cost $\Theta(L)$. Probe count alone omits key-processing time.
- An attacker controlling many colliding keys can invalidate distribution assumptions. Possible responses include random seeds, suitable keyed hashing, or another container.
- Sorted iteration, range queries, and predecessors are more direct with balanced search trees, since hash tables do not preserve key order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-stu

References and further reading

- The MIT 6.006 2008 index, L5–L7, identifies hashing course material.
- C++ unordered associative requirements distinguish average from worst-case costs and require compatible hash and equality functions. Average guarantees do not mean worst-case constant time for every call.
- MIT 6.006 2014 Recitation 10, § 1.1: Deleted search semantics and duplicate-key updates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study